SMPTE Public Committee Draft

Catena — Protocol Objects

This material is work under development and shall not be referred to as a SMPTE Standard, Recommended Practice, or Engineering Guideline. It is distributed for review and comment; distribution does not constitute publication.

Please be aware that all contributions to this material are being conducted in accordance with the SMPTE Standards Operations Manual, which is accessible on the SMPTE website with the Society Bylaws:

https://www.smpte.org/about/policies-and-governance

Your comments and contributions, whether as a member or guest, are governed by these provisions and any comment or contribution made by you indicates your acknowledgement that you understand and are complying with the full form of the Operations Manual. Please take careful note of the clauses requiring contributors to inform the Committee of personal knowledge of any claims under any issued patent or any patent application that likely would be infringed by an implementation of this material. This general reminder is not a substitute for a contributor's responsibility to fully read, understand, and comply with the full Standards Operations Manual.

Copyright Notice

Copyright © 2026 by the Society of Motion Picture and Television Engineers. All rights reserved. No part of this material may be reproduced, by any means whatsoever, without the prior written permission of the Society of Motion Picture and Television Engineers.

Patent Notice

Attention is drawn to the possibility that some of the elements of this material may be the subject of patent rights. SMPTE shall not be held responsible for identifying any or all such patent rights.

A list of all public CDs can be found on the SMPTE website

https://www.smpte.org/public-committee-drafts#listing

For review only

Table of Contents Page

Foreword 3
Introduction 3
1 Scope 3
2 Conformance 3
3 Normative References 4
4 Terms and Definitions 4
5 Precedence 4
6 Protocol Objects 4
6.1 Overview 4
6.2 Slot List 5
6.3 Device Component 5
6.4 Execute Command and Command Response 6
6.5 External Object Payload 7
6.6 Data Payload 7
6.7 Param Info 8
6.8 Component Param 9
6.9 Push Updates 10
6.10 Languages 11
6.11 Access Control 11
6.12 Value and Values 12
Annex A (Informative) Additional Elements 13

For review only

 © SMPTE 2026 – All Rights Reserved

Foreword

SMPTE (the Society of Motion Picture and Television Engineers) is an internationally-recognized standards developing organization. Headquartered and incorporated in the United States of America, SMPTE has members in over 80 countries on six continents. SMPTE's Engineering Documents, including Standards, Recommended Practices, and Engineering Guidelines, are prepared by SMPTE's Technology Committees. Participation in these Committees is open to all with a bona fide interest in their work. SMPTE cooperates closely with other standards-developing organizations, including ISO, IEC and ITU.

SMPTE Engineering Documents are drafted in accordance with the rules given in its Standards Operations Manual. This SMPTE Engineering Document was prepared by Technology Committee TC34CS – Media Systems, Control and Services.

Introduction

This clause is entirely informative and does not form an integral part of this Engineering Document.

SMPTE ST 2138 envisages the use of several API frameworks, including REST and gRPC. The objects that are sent from a service or device to a connected client are usually independent of the framework, and so they are documented here where each API can reference the protocol object definitions.

1 Scope

This document defines the objects that are exchanged between participants using the SMPTE ST 2138 protocol.

2 Conformance

Normative text is text that describes elements of the design that are indispensable or contains the conformance language keywords: "shall", "should", or "may". Informative text is text that is potentially helpful to the user, but not indispensable, and can be removed, changed, or added editorially without affecting interoperability. Informative text does not contain any conformance keywords.

All text in this document is, by default, normative, except: the Introduction, any clause explicitly labeled as "Informative" or individual paragraphs that start with "Note:"

The keywords "shall" and "shall not" indicate requirements strictly to be followed in order to conform to the document and from which no deviation is permitted.

The keywords, "should" and "should not" indicate that, among several possibilities, one is recommended as particularly suitable, without mentioning or excluding others; or that a certain course of action is preferred but not necessarily required; or that (in the negative form) a certain possibility or course of action is deprecated but not prohibited.

The keywords "may" and "need not" indicate courses of action permissible within the limits of the document.

The keyword "reserved" indicates a provision that is not defined at this time, shall not be used, and may be defined in the future. The keyword "forbidden" indicates "reserved" and in addition indicates that the provision will never be defined in the future.

A conformant implementation according to this document is one that includes all mandatory provisions ("shall") and, if implemented, all recommended provisions ("should") as described. A conformant implementation need not implement optional provisions ("may") and need not implement them as described.

Unless otherwise specified the order of precedence of the types of normative information in this document shall be as follows. Normative prose shall be the authoritative definition. tables shall be next, followed by formal languages, then figures, and then any other language forms.

For review only

© SMPTE 2026 – All Rights Reserved

3 Normative References

The following standard contains provisions that, through reference in this text, constitute provisions of this standard. Dated references require that the specific edition cited shall be used as the reference. Undated citations refer to the edition of the referenced document (including any amendments) current at the date of publication of this document. All standards are subject to revision, and users of this engineering document are encouraged to investigate the possibility of applying the most recent edition of any undated reference.

SMPTE ST 2138-10:202x, *Catena – Model*

ISO 639:2023, *Code for individual languages and language groups, edition 2, 2023*.

4 Terms and Definitions

No terms and definitions are listed in this document.

5 Precedence

In the event of a conflict between the schema located at additional element at https://github.com/SMPTE/st2138-a.git (in the /interface/schemata/device.yaml file) and other information in this document, the schema shall take precedence. See Annex A for additional information.

6 Protocol Objects

6.1 Overview

Each protocol object is defined in a self-named subclause in this document. Formal definitions are provided by Protobuf message definitions, some of which are augmented by JSON-schema and cited by reference to publicly available resources throughout this document as appropriate.

- Protobuf filenames are relative to https://github.com/SMPTE/st2138-a/tree/main/interface/proto, they have `.proto` as their extension.
- JSON-schemata are relative to https://smpte.github.io/st2138-a/interface/schemata/device.yaml, the schemata are derived from the way Protobuf serializes its messages to JSON; as such, they are usually a secondary part of the protocol definition[1].
- Protobuf does not support constraining map keys to match regular expressions. In some cases, the ability of JSON-schema to have string objects conform to a regular expression has been used. Normally, it is best to limit the names of objects to a character set that enables code generators to easily create type definitions and object names that are legal in the targeted programming language, and also to limit the length of parameter arguments that permit input validation and defend against one form of denial-of-service attack. In these cases, the JSON schema reinforces the Protobuf definition and is thus a part of the primary protocol definition.

[1] The length of the file is approximately 3000 lines. A convenient way to find a schema is to search for it by name, followed by the colon character. This will usually result in only one or two hits, making it straightforward to find the schema definition whereas searching by name alone will also find all the places that reference the schema. EXAMPLE: Search term for the `slot_list` schema is "`slot_list:`".

 © SMPTE 2026 – All Rights Reserved

For review only

- Although Protobuf 2.0 supported marking members of messages as required, this feature was deprecated in Protobuf 3.0, which is the version used for this interface definition. So, JSON-schema's ability to mark fields as required is used in the schemata definitions, another way in which the JSON-schema reinforces the primary Protobuf protocol definition. In the UML representations of the protocol objects, the stereotype `<<required>>` indicates when a field must be included in the communicated object.

6.2 Slot List

Slot List identifies which slots contain device models, as illustrated in Figure 1:

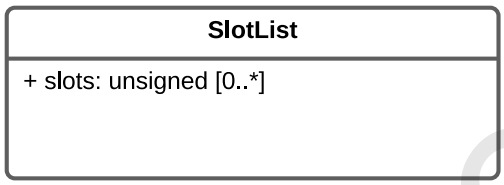

Figure 1 — Slot List

- Protobuf message `SlotList` is defined in `device.proto`.
- Schema: `#/$defs/slot_list`

6.3 Device Component

The `DeviceComponent` may be a `ComponentParam`, a `ComponentLanguagePack`, a `ComponentMenu`, a `ComponentConstraint`, or a `ComponentCommand`, as shown in Figure 2.

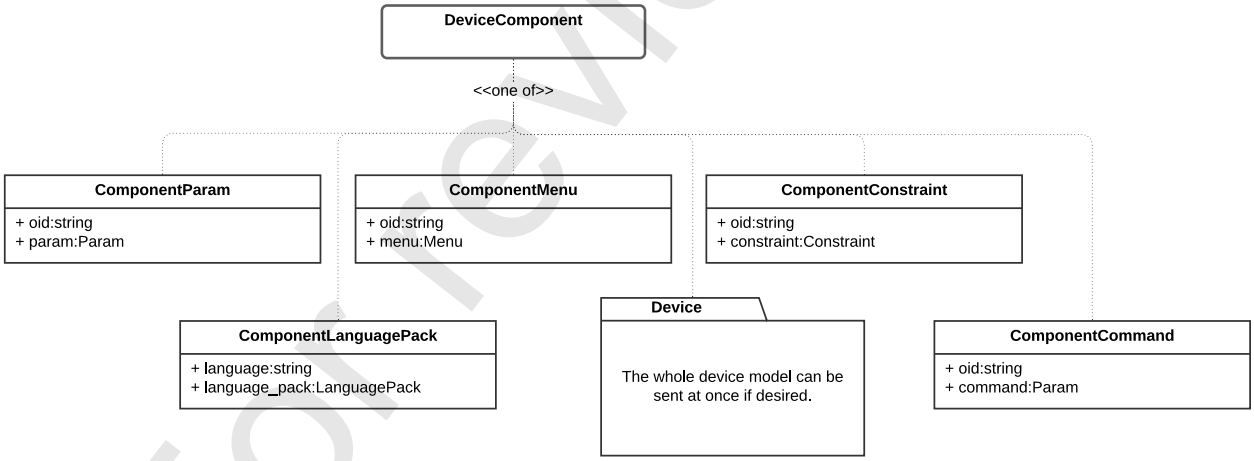

Figure 2 — Device Component

The `oid` strings in the objects illustrated are instances of the `simple_oid` schema which limits their length and characters set via a regular expression.

The `language` string has a maximum length of 32.

- Protobuf message `DeviceComponent` is defined in `device.proto`, as are its polymorphisms.
- Schema: `#/$defs/device_component`

For review only

6.4 Execute Command and Command Response

The Execute Command payload as shown in Figure 3 includes an optional value field for commands that take values; for example, a Play command may be accompanied by a value for Play Speed. The Protobuf `ExecuteCommandPayload` message is defined in `param.proto`, the schema is at `#/$defs/execute_command_payload`.

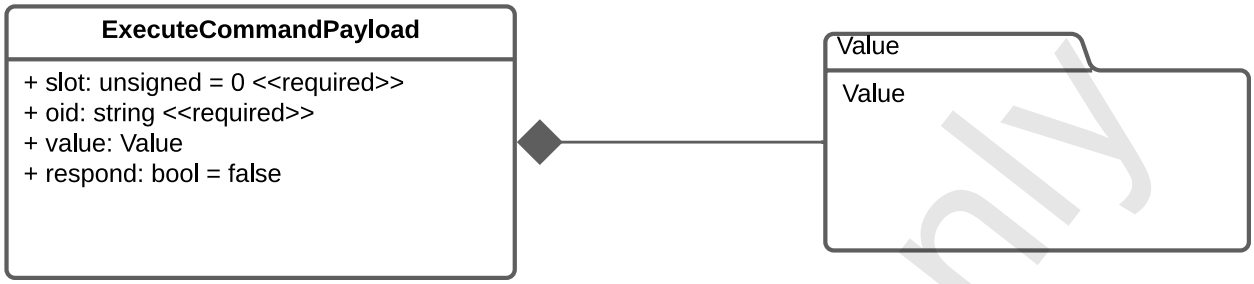

Figure 3 — Execute Command Payload

A Command Response may contain no response, a response, or an exception, as shown in Figure 4.

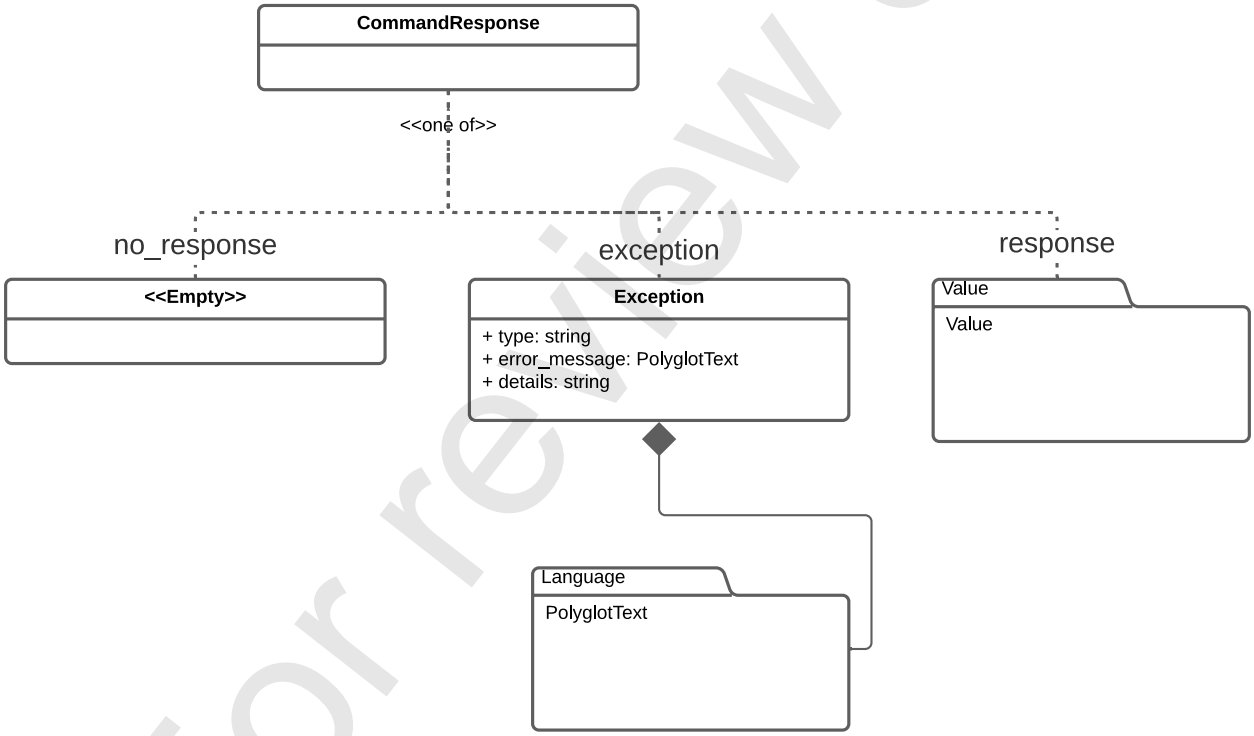

Figure 4 — Command Response

The Protobuf `CommandResponse` message is defined in `param.proto`, and the schema is at `#/$defs/command_response`.

For review only

 © SMPTE 2026 – All Rights Reserved

6.5 External Object Payload

This is a Data Payload with a flag to indicate whether the message can be cached, as shown in Figure 5.

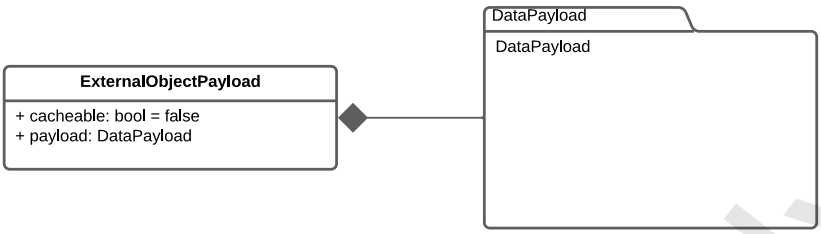

Figure 5 — External Object Payload

The Protobuf message is defined in `externalobject.proto`, and `#/$defs/external_object_payload`.

NOTE In the REST API, “External Objects” are called “Assets”.

6.6 Data Payload

The Data Payload may contain a URL or the payload itself, encoded in one of three ways, as shown in Figure 6.

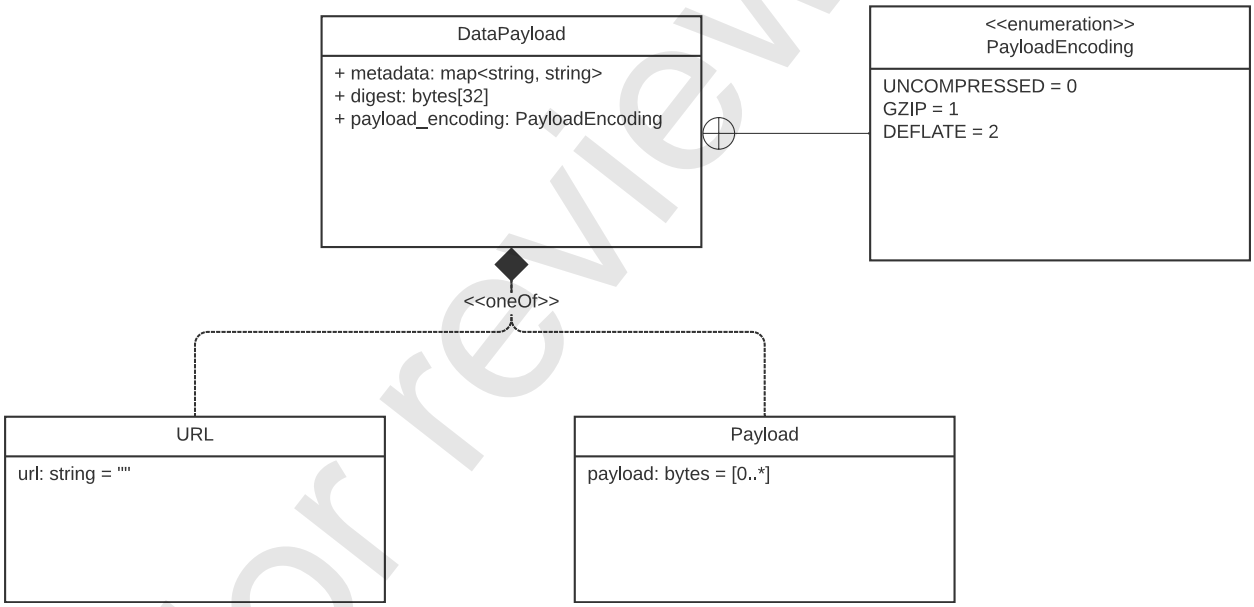

Figure 6 — Data Payload

When the Data Payload comprises raw data rather than a URL, the SHA-256 digest shall be provided in the response.

Optional key-value metadata may also be part of the response. Examples include: filename and MIME type.

The Protobuf message `DataPayload` is defined in `param.proto` and the schema is at `#/$defs/data_payload`.

For review only

6.7 Param Info

The gRPC API uses `ParamInfo` which is adapted to `param-info` for the REST API, as shown in Figure 7.

NOTE "param-info" is used for the REST-API.

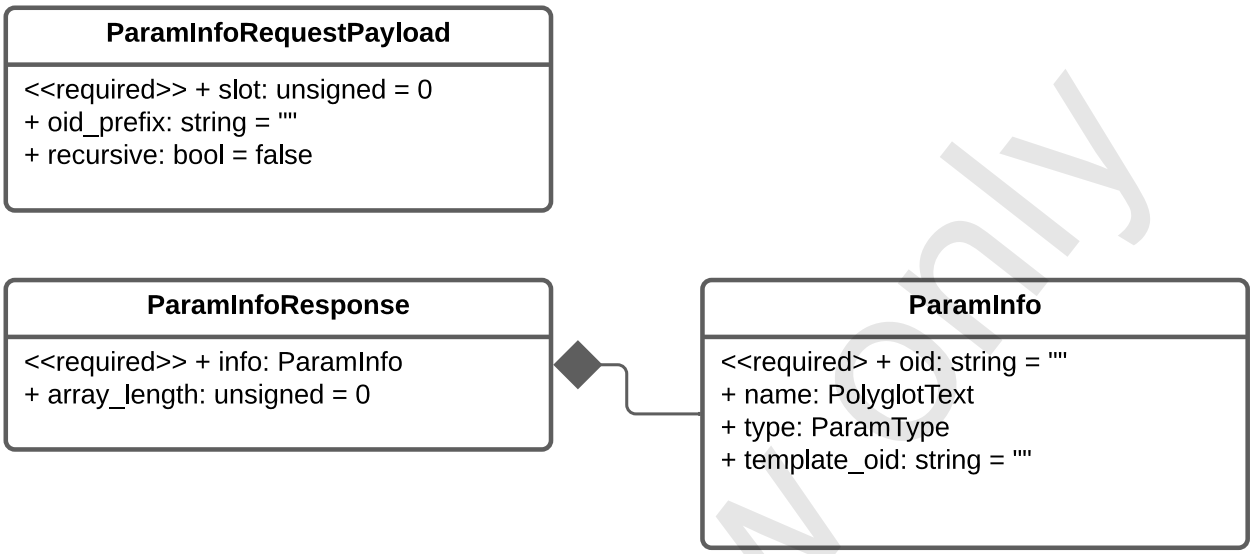

Figure 7 — Param Info

The `oid_prefix` is an instance of the `fqoid` schema which specifies valid values by means of a regular expression. `fqoid` objects are fully qualified object ids and permit parameters inside a deep hierarchy to be addressed.

When `recursive` is set, the parameter's children shall be included in the response.

In the response, if the parameter is an array type, the length of the array shall be returned in addition to the `info` field.

- Protobuf messages for `ParamInfoRequestPayload`, `ParamInfoResponse` and `ParamInfo` are in `param.proto`.
- Schemata: `#/$defs/param_info_request`, `#/$defs/param_info_response`, `#/$defs/param_info`.

For review only

 © SMPTE 2026 – All Rights Reserved

6.8 Component Param

The Component Param structure is shown in Figure 8.

The Param Object is defined in SMPTE ST 2138-10:202x *Catena — Model*, Clause 9.7.

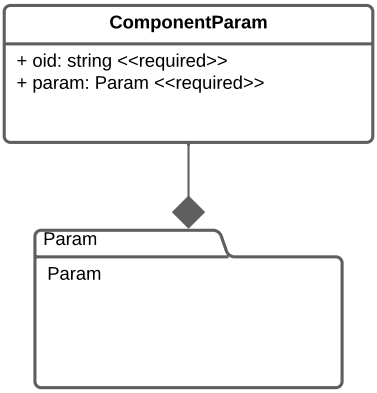

Figure 8 — Component Param

Protobuf `Param` message is defined in `param.proto`.

Protobuf `ComponentParam` message is defined in `device.proto`.

Schemata: `#/$defs/param`, `#/$defs/component_param`

For review only

© SMPTE 2026 – All Rights Reserved

6.9 Push Updates

Push Updates may specify either an Exception, Remove Components, or a Trap Message, as shown in Figure 9 and in Table 1.

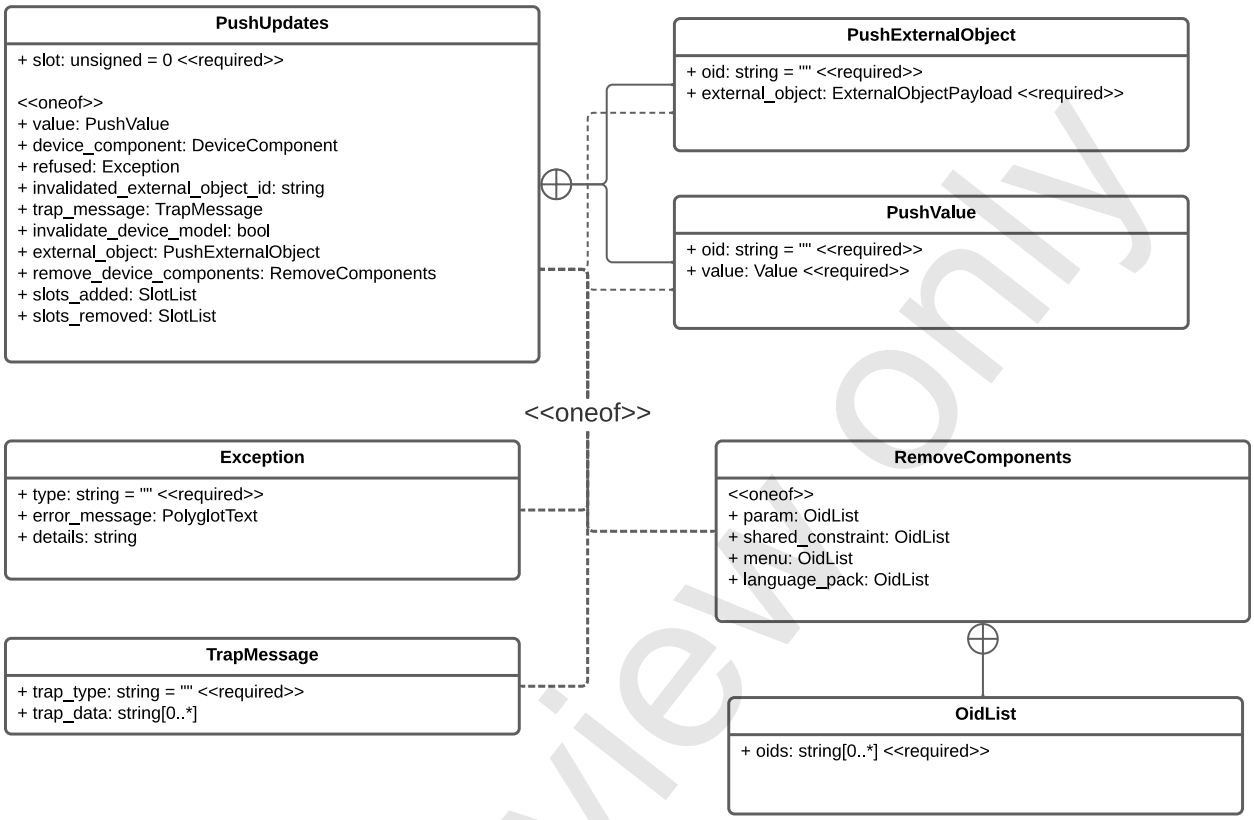

Figure 9 — Push Updates Class Diagram

Table 1 — Protobuf Message Definitions

Object	Protobuf Message Defined in	Schema
PushUpdates	device.proto	#/$defs/push_updates
Exception	param.proto	#/$defs/exception
TrapMessage	device.proto	#/$defs/trap_message
PushExternalObject	device.proto	#/$defs/push_external_objec t
PushValue	device.proto	#/$defs/push_value
RemoveComponents	device.proto	#/$defs/remove_components

For review only

 © SMPTE 2026 – All Rights Reserved

6.10 Languages

`LanguageList` provides a list of allowed languages, as shown in Figure 10.

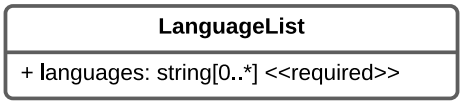

Figure 10 — Languages List

The LanguageList object is used by a device to report which languages it supports. Each string is constrained by a regular expression to conform to the ISO-639 format, this does not rule out invalid codes such as “zz-ZZ” but does validate good ones such as “en”, and “en-CA”. The schema is `#/$defs/language_code`.

The `LanguageList` Protobuf message is defined in `language.proto`, its schema is `#/$defs/language_list`

6.11 Access Control

Access control is implemented by the device interpreting the access token that is attached to each request. Access tokens should have a brief lifespan of around 3 minutes. They can be refreshed by a connected client that anticipates token expiry and obtains a new token in advance.

The new token is attached to the request to refresh access to the device as metadata, so does not appear in the object sent as the request at the protocol layer.

The Protobuf `RefreshTokenPayload` is defined in `device.proto`, as is the `ConnectionStatus` message as shown in Figure 11.

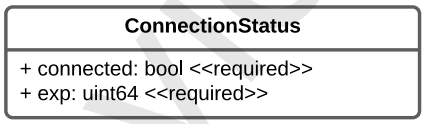

Figure 11 — Connection Status Object

Schemata: `#/$defs/refresh_token_payload` and `#/$defs/connection_status`.

Revocation is an important aspect of access control, and it is supported by the protocol objects, shown in Figure 12.

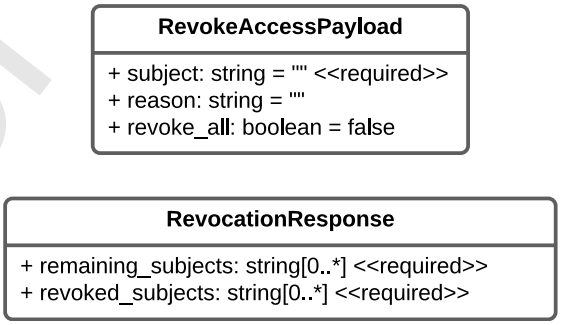

Figure 12 — Revocation Class Diagram

The Protobuf `RevokeAccessPayload` message is defined in `device.proto`, as is the `RevocationResponse` message.

Schemata: `#/$def/revoke_access_payload`, `#$def/revocation_response`

For review only

© SMPTE 2026 – All Rights Reserved

6.12 Value and Values

The Value object is defined in ST 2138-10:202x Clause 9.8. Some transactions wrap additional information around Value objects, as illustrated in Figure 13.

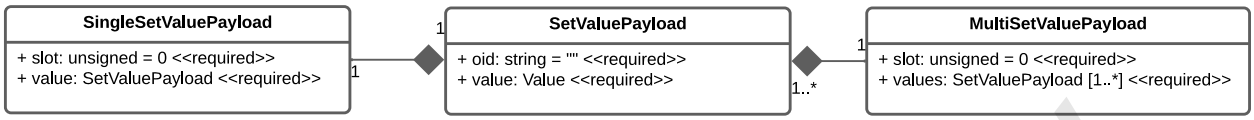

Figure 13 — Set Value Payloads

The Protobuf `SingleSetValuePayload` and `MultiSetValuePayload` messages are defined in `param.proto`.

For review only

 © SMPTE 2026 – All Rights Reserved

Annex A
(Informative)

Additional Elements

Repository

A set of schemas represented by the UML diagrams in this document provided as a companion element to this document. These can be found in the public repository at the following URL:

https://github.com/SMPTE/st2138-a.git

The software version corresponding to this document revision is tagged as SMPTE-ST-2138-19-2026.

For review only

© SMPTE 2026 – All Rights Reserved